



SMIT SIKKIM
MANIPAL
UNIVERSITY
SIKKIM MANIPAL INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER APPLICATIONS

Majitar, Rangpo, East Sikkim – 737136

ASSIGNMENT REPORT

Subject: .NET Programming • **Semester:** 5th Semester (BCA)

Console-Based Tic-Tac-Toe Game

Github: <https://github.com/skysys007/Tic-Tac-Toe>

Submitted By:

Gagan Pradhan
Reg. No: **202416006**
BCA 5th Semester

Ranit Mahato
Reg. No: **202416051**
BCA 5th Semester

Contents

1	Introduction	1
2	System Requirements	1
3	Code Implementation & Analysis	1
3.1	Global State Variables	1
3.2	The Main Game Loop	2
3.3	Rendering the Board	3
3.4	Win Detection	4
3.5	Draw Detection	5
4	Screenshots	6
5	Conclusion	7

Explanation:

- **board:** A character array of size 9 representing the 3×3 grid. Initialized with empty spaces.
- **position:** An integer tracking the user's cursor index (0 to 8). It defaults to 4 (the center position).
- **player:** A character tracking the active player. It alternates between 'X' and 'O' upon each successful turn.

3.2 The Main Game Loop

The `Main` method orchestrates the game's execution, capturing user input and verifying game states.

```
1  static void Main()
2  {
3      while (true)
4      {
5          Console.Clear();
6          ShowBoard();
7
8          Console.WriteLine("WASD / Arrow Keys = Move | Enter = Place
9          " + player);
10         ConsoleKey key = Console.ReadKey(true).Key;
11
12         // Movement controls
13         if (key == ConsoleKey.W || key == ConsoleKey.UpArrow)
14             position -= 3;
15         if (key == ConsoleKey.S || key == ConsoleKey.DownArrow)
16             position += 3;
17         if (key == ConsoleKey.A || key == ConsoleKey.LeftArrow)
18             position--;
19         if (key == ConsoleKey.D || key == ConsoleKey.RightArrow)
20             position++;
21
22         // Boundary constraints
23         if (position < 0) position = 0;
24         if (position > 8) position = 8;
25
26         if (key == ConsoleKey.Enter)
27         {
28             if (board[position] == ' ')
29             {
30                 board[position] = player;
```

```
27         if (Winner())
28         {
29             Console.Clear(); ShowBoard();
30             Console.WriteLine(player + " Wins!"); break;
31         }
32
33         if (Draw())
34         {
35             Console.Clear(); ShowBoard();
36             Console.WriteLine("Draw!"); break;
37         }
38
39         player = player == 'X' ? 'O' : 'X';
40     }
41 }
42 }
43 }
```

Listing 2: Main Method Loop

Explanation:

The loop continuously refreshes the console, displaying the updated board. It intercepts keyboard presses, updating the `position` variable based on directional inputs (WASD or arrows). When the user presses **Enter**, it verifies if the target tile is empty. If so, it assigns the tile to the current player and subsequently triggers the `Winner()` and `Draw()` checks to determine if the game has concluded.

3.3 Rendering the Board

The `ShowBoard` method translates the internal array state into a visual grid on the console.

```
1  static void ShowBoard()
2  {
3      for (int i = 0; i < 9; i++)
4      {
5          if (i == position) Console.Write("[");
6          else Console.Write(" ");
7
8          if (board[i] == 'X') Console.ForegroundColor = ConsoleColor
.Red;
9          else if (board[i] == 'O') Console.ForegroundColor =
ConsoleColor.Blue;
10
11         Console.Write(board[i]);
12         Console.ResetColor();
13     }
```

```

14         if (i == position) Console.Write("]");
15         else Console.Write(" ");
16
17         if (i % 3 == 2) Console.WriteLine();
18         else Console.Write("|");
19     }
20 }

```

Listing 3: Board Rendering Logic

Explanation:

This method iterates through the 9 tiles of the board. It highlights the currently selected `position` using bracket cursors `[ ]`. Furthermore, it dynamically alters the foreground color (Red for 'X', Blue for 'O') for better visual distinction. Newlines and pipe characters `|` are strategically printed to format the 1D array into a 2D 3×3 grid structure.

3.4 Win Detection

The `Winner` method cross-references the board state against all valid winning patterns.

```

1  static bool Winner()
2  {
3      int[,] win =
4      {
5          { 0, 1, 2 }, { 3, 4, 5 }, { 6, 7, 8 }, // Horizontal
6          { 0, 3, 6 }, { 1, 4, 7 }, { 2, 5, 8 }, // Vertical
7          { 0, 4, 8 }, { 2, 4, 6 }                // Diagonal
8      };
9
10     for (int i = 0; i < 8; i++)
11     {
12         if (board[win[i, 0]] == player &&
13             board[win[i, 1]] == player &&
14             board[win[i, 2]] == player)
15             return true;
16     }
17     return false;
18 }

```

Listing 4: Win Checking Algorithm

Explanation:

A 2D array `win` maintains the 8 possible winning combinations (3 rows, 3 columns, 2 diagonals). The method loops through these combinations and checks if the current player's mark is present at all three indices of any given combination. If a match is found, it evaluates to `true`, signaling a victory.

3.5 Draw Detection

If the board fills up without any winning combination being met, a draw is declared.

```
1  static bool Draw()  
2  {  
3      for (int i = 0; i < 9; i++)  
4      {  
5          if (board[i] == ' ' )  
6              return false;  
7      }  
8      return true;  
9  }
```

Listing 5: Draw Condition Evaluation

Explanation:

This function acts as a fallback checker. It inspects all 9 elements in the `board` array. If it discovers any empty space (' '), it confirms that moves are still possible and returns `false`. If no spaces are empty (and a win hasn't occurred), it returns `true`, successfully identifying a draw.

4 Screenshots

The following screenshots display the game in various states of play.

```

skysys@Swift: ~/Desktop/Tic-tac-toe/src
| | |
| | |
| | |
WASD / Arrow Keys = Move | Enter = Place X

```

(a) Initial game start

```

skysys@Swift: ~/Desktop/Tic-tac-toe/src
| | |
[0] | X |
| | |
WASD / Arrow Keys = Move | Enter = Place X

```

(b) Mid-game progression

```

skysys@Swift: ~/Desktop/Tic-tac-toe/src
X | X |
O | X |
O | [X] | O
X Wins!
skysys@Swift:~/Desktop/Tic-tac-toe/src/TicTa

```

(c) Player interaction

```

skysys@Swift: ~/Desktop/Tic-tac-toe/src
O | | 
X | O |
X | X | [O]
O Wins!
skysys@Swift:~/Desktop/Tic-tac-toe/src/TicTa

```

(d) End game state

Figure 1: Gameplay Snapshots

5 Conclusion

This assignment successfully implements a complete, playable Tic-Tac-Toe game in C#. By organizing the game logic into dedicated methods and maintaining a clear separation of concerns, the code remains highly readable, structured, and easily extensible for future enhancements, such as integrating an AI opponent or migrating to a graphical user interface. The implementation effectively meets all functional requirements while keeping the codebase compact, efficient, and thoroughly documented.